

Prodotto Matrice Densa-Multivettore

SCPA Project A.Y. 2025/2026

Matteo Avallone
dept. Software Engineering
University of Rome, Tor Vergata
Rome, Italy
matteo.avallone2000@gmail.com

Abstract—Il presente progetto si propone di implementare e analizzare un nucleo di calcolo tra una matrice densa A e un multi-vettore X ($Y = AX$), con $M \times N$ e $N \times k$ dimensioni di A e X rispettivamente. A tal fine, è stata adottata una griglia di processi bidimensionale e la parallelizzazione dei compiti è stata realizzata mediante MPI e CUDA. Diversi Kernel CUDA sono stati testati su diverse configurazioni di A e X (valori di M , N e k) e successivamente confrontati con una versione seriale del codice sviluppato.

La sezione conclusiva della relazione sarà dedicata all'analisi dei risultati ottenuti, presentati graficamente, al fine di evidenziare la differenza di prestazioni rispetto ad un'implementazione del sistema basata sull'efficiente libreria cuBLAS.

I. INTRODUZIONE

Il prodotto tra matrice densa e multi-vettore è un'operazione fondamentale largamente utilizzata in vari domini applicativi, quali:

- Computer graphics
- Deep learning
- Analisi numerica

Con l'emergere di numerosi ed eterogenei sistemi di High Performance Computing (HPC) dotati di molteplici unità di elaborazione grafica (GPU), risulta fondamentale analizzare strategie efficaci per l'utilizzo di tali acceleratori, con l'obiettivo di ottimizzare l'efficienza del calcolo numerico.

Il presente progetto si focalizzerà sul design e l'ottimizzazione iterativa del problema in esame, sfruttando la combinazione di Message Passing Interface (MPI) e CUDA. L'applicazione implementata sarà in grado di calcolare $Y=AX$, dove:

$$A \in \mathbb{R}^{M \times N}, X \in \mathbb{R}^{N \times k}, Y \in \mathbb{R}^{M \times k}$$

A differenza della moltiplicazione standard matrice-vettore, l'impiego di un multi-vettore consente un incremento del parallelismo e permette l'esplorazione di differenti strategie di ottimizzazione in funzione della dimensione del problema.

Il progetto si propone di integrare due livelli di parallelismo:

1. **Parallelismo inter-nodo:** realizzato tramite MPI, utilizzando una griglia 2D di processi per la distribuzione dei blocchi di matrice tra i nodi;
2. **Parallelismo intra-nodo:** realizzato tramite l'esecuzione di vari kernel CUDA eseguiti su GPUs NVIDIA.

L'obiettivo chiave di questo lavoro è pertanto lo sviluppo di un sistema adattivo di assegnazione kernel capace di

selezionare il kernel GPU più adeguato a seconda delle dimensioni della matrice/multi-vettore e del carico di lavoro complessivo. L'implementazione esplora differenti tecniche di ottimizzazione GPU come:

- Shared memory tiling
- Warp-level parallelism
- Register blocking
- Memory coalescing/Vectorized loads

I risultati ottenuti sul server GPU del dipartimento verranno successivamente confrontati numericamente con un'esecuzione seriale di ciascuna configurazione, al fine di verificarne la precisione numerica. Infine, verrà eseguito un confronto con un kernel cuBLAS per quantificare l'efficienza ed identificare eventuali colli di bottiglia nella performance.

II. METRICHE

Per valutare l'efficacia del progetto realizzato, sono state usate numerose metriche di performance e correttezza durante il processo di analisi sperimentale.

A. GFLOPS

La metrica principale è GFLOPS (Giga Floating Point Operations Per Second). Misura quante operazioni floating-point vengono eseguite al secondo dall'implementazione.

$$GFLOPS = \frac{2MNk}{T * 10^9}$$

dove T è il tempo medio di esecuzione del programma.

B. Speed Up

Questa metrica valuta quanto più veloce è risultata l'esecuzione parallela MPI+CUDA rispetto alla baseline seriale su CPU. Definita come:

$$Speedup = \frac{T_{serial}}{T_{parallel}}$$

C. Errore Assoluto

Si tratta di una metrica utile per rilevare grosse deviazioni numeriche tra le due implementazioni. L'errore assoluto per ogni elemento in output è definito come:

$$e_{abs} = |y_{serial} - y_{parallel}|$$

Mentre il massimo errore assoluto tra tutti gli elementi in output è:

$$E_{abs}^{max} = \max_i |y_i^{serial} - y_i^{parallel}|$$

D. Errore Relativo

Poiché i valori in virgola mobile possono variare notevolmente in ordine di grandezza, l'errore assoluto potrebbe risultare fuorviante. È stata pertanto introdotta una metrica di errore relativo per normalizzarne le discrepanze:

$$e_{rel} = \frac{|y_{serial} - y_{parallel}|}{|y_{serial}|}$$

Fornisce una misura dell'accuratezza numerica indipendente dalla scala, escludendo derive numeriche distruttive introdotte dal parallelismo massivo della GPU o dall'ordine non associativo delle operazioni aritmetiche.

E. Efficienza rispetto a cuBLAS

Metrica utilizzata per quantificare la qualità dell'implementazione proposta rispetto a cuBLAS, baseline professionale e ottimizzata per specifiche architetture:

$$\text{Efficiency} = \frac{\text{GFLOPS}_{\text{custom}}}{\text{GFLOPS}_{\text{cuBLAS}}} \cdot 100$$

F. Overhead di comunicazione

Dato che l'applicazione è distribuita tra processi MPI, l'overhead di comunicazione non è stato incluso nei GFLOPs ma misurato separatamente.

- Tempo di trasferimento Host-to-Device (H2D)
- Tempo di trasferimento Device-to-Host (D2H)
- Tempo di comunicazione MPI.

III. DATA PREPROCESSING

3.1 Griglia dei Processi

Come richiesto da specifica, al fine di distribuire la moltiplicazione tra più processi, l'applicazione organizza i rank MPI in una griglia dei processi bidimensionale e cartesiana, di taglia $P \times Q$ (numero di processi MPI). La griglia viene inizializzata dalla funzione *setup_process_grid()*, che va innanzitutto a verificare che il numero totale di processi MPI corrisponda alle dimensioni richieste della griglia.

Grazie alla funzione *MPI_Cart_create()*, costruiamo un communicator cartesiano bidimensionale; a ciascun processo vengono assegnate le coordinate (i, j) all'interno della griglia, semplificando così le operazioni di partizionamento della matrice e comunicazione collettiva.

Al fine di ridurre l'overhead di comunicazione, sono stati creati due sub-communicators:

- **Row communicator:** raggruppa i processi appartenenti alla stessa riga della griglia;
- **Column communicator:** raggruppa i processi appartenenti alla stessa colonna della griglia.

Ciò è reso possibile grazie a *MPI_Cart_sub()*; in questo modo, le operazioni collettive avverranno solo tra sottoinsiemi rilevanti dei processi, invece dell'intero comunicatore.

3.2 Matrix Generation

Per generare la matrice A in maniera riproducibile e deterministica, è stata utilizzata una leggera rivisitazione della formula alla base delle matrici di Hilbert.

Una matrice di Hilbert è una matrice caratterizzata da componenti che dipendono esclusivamente dagli indici i e j. Per l'esecuzione seriale, ciascun processo genera il proprio blocco di matrice locale utilizzando le coordinate della matrice globale:

$$A_{ij} = \frac{1}{1 + i + j}$$

dove i e j sono, rispettivamente, gli indici globali di riga e colonna. Per l'esecuzione parallela, ciascun rank MPI calcola la propria sottomatrice tramite l'ausilio di *row_offset* e *col_offset*, per mappare indici locali a coordinate globali e basandosi pertanto esclusivamente sulla propria posizione nella griglia dei processi:

$$A_{ij} = \frac{1}{1 + (i + \text{row_offset}) + (j + \text{col_offset})}$$

Questa implementazione distribuita della matrice A consente di evitare il ricorrere alla comunicazione per l'inizializzazione, rendendo il processo di costruzione della matrice scalabile. Inoltre, le matrici di Hilbert sono note per la loro proprietà di "ill-conditioning" che cresce all'aumentare della loro taglia; ciò può portare a problemi di discrepanze numeriche anche considerevoli durante i calcoli con numeri in virgola mobile. Pertanto, la scelta di una tale matrice è stata dettata dal voler fornire un test deterministico utile per valutare la consistenza delle operazioni numeriche tra il codice seriale e il parallelo.

Anche il multi-vettore X è stato generato mediante una formula deterministica basata sull'espressione analitica:

$$X_{jc} = \sin(j + c)$$

con j indice di riga globale e c indice di colonna; per il calcolo parallelo, ciascun processo sostituisce j con $j + \text{col_offset}$.

3.3 Matrix Distribution

La matrice A, di dimensioni $M \times N$, è distribuita a blocchi 2D: ogni processo possiede un sottoblocco contiguo di dimensioni $M_loc \times N_loc$, dove:

- $M_loc = M/P$ (con le righe extra distribuite ai primi processi)
- $N_loc = N/Q$ (analogamente per le colonne).

Il processo (i, j) gestisce le righe nell'intervallo $[\text{row_offset}, \text{row_offset} + M_loc)$ e le colonne $[\text{col_offset}, \text{col_offset} + N_loc)$. I valori del blocco locale vengono generati da *generate_A_block()*, evitando comunicazione per la distribuzione di A. Questa strategia, infatti, riduce il memory footprint da $O(MN)$ ad approssimativamente

$$O\left(\frac{MN}{PQ}\right)$$

Il multi-vettore X, di dimensioni $N \times k$, è distribuito per righe lungo la dimensione delle colonne della griglia. Ogni

processo possiede $X_local.local_size$ righe di X , corrispondenti alle stesse colonne di A che possiede. La distribuzione avviene tramite *MPI_Scatterv* sul *row_comm*, pertanto solo i processi con *my_col* == 0 generano X globale in un buffer temporaneo; *sendcounts[q]* e *displs_x[q]* vengono calcolati con gestione del resto ($N \% Q$). Ogni processo riceverà, pertanto, esattamente le $N_loc \times k$ entry richieste per il prodotto con il proprio blocco locale di A . Questo approccio riduce la comunicazione ad un $O(N \times k / Q)$ per processo, rispetto ad un $O(Nk)$ di una *MPI_Bcast*.

Infine, anche il multi-vettore Y risultante, di dimensioni $M \times k$, viene distribuito per righe lungo la dimensione delle righe della griglia. Ogni processo alloca $M_loc \times k$ elementi, dove M_loc e *row_offset* sono ereditati dalla struttura locale di A .

IV. BASELINE SERIALE E VERIFICA NUMERICA

La correttezza dell'implementazione parallela MPI+CUDA è stata validata rispetto ad una implementazione seriale di riferimento, eseguita sulla CPU. Il prodotto viene eseguito, in questo caso, seguendo il classico loop con tre annidamenti, su indici di M , N e k . Ciascun elemento di output viene calcolato in maniera indipendente e costituisce una verità di base per validare l'accuratezza dei risultati numerici ottenuti dal calcolo parallelo. La complessità di questo algoritmo è $O(MNk)$, pertanto particolarmente dispendioso per matrici di grandezza considerevole.

La routine di verifica numerica adotta un criterio di accettazione duale: un elemento è considerato errato solo se contemporaneamente supera sia la soglia assoluta *abs_t* che quella relativa *rel_t*;

$$\begin{aligned} \text{errore} &\Leftrightarrow |ref - got| \\ &> abs_t \text{ AND } |ref - got| \\ &> \max(|ref|, |got|) > rel_t \end{aligned}$$

Questa scelta previene falsi positivi che emergono con un criterio singolo di verifica. Un errore puramente assoluto sarebbe troppo severo per valori grandi (dove piccole differenze sono attese per via dell'aritmetica floating-point), mentre un errore puramente relativo sarebbe troppo severo vicino allo zero (dove differenze minuscole producono rapporti grandi). Per gli elementi il cui valore di riferimento è vicino allo zero, il denominatore del calcolo relativo viene settato a 1.0 quando $\max(|ref|, |got|) < 1e-7$; questo evita divisioni per zero o rapporti artificialmente gonfiati su elementi quasi nulli, riportandoli di fatto ad un confronto puramente assoluto.

V. MPI

L'implementazione del progetto si appoggia su MPI per la partizione dei dati, la coordinazione tra i processi e il raggruppamento dei risultati parziali.

A. Sincronizzazione tra processi e Timing

Le seguenti routine MPI sono state utilizzate per coordinare l'esecuzione e misurare la performance generale:

- *MPI_Wtime()*: utilizzata per ottenere "wall-clock timings" (tempo trascorso in secondi) per le fasi di setup, comunicazione e computazione.
- *MPI_Barrier()*: utilizzata per sincronizzare tutti i processi prima di entrare nelle sezioni critiche, assicurando misurazioni temporali consistenti. Ad esempio, una barriera è stata inserita dopo la fase di distribuzione dei dati in modo da consentire a tutti i processi di iniziare la computazione simultaneamente.

B. Distribuzione di X

Mentre ciascun processo genera in maniera indipendente il proprio blocco locale della matrice A , il multi-vettore X deve essere distribuito tra le colonne dei processi. Al fine di distribuirlo, è stata impiegata la routine *MPI_Scatterv* che, a differenza della variante *MPI_Scatter* la quale richiede chunk di taglia eguale, supporta varie taglie di messaggi tramite:

- *sendcounts*, che specifica il numero di elementi inviati a ciascun processo;
- *displs*, che specifica la posizione iniziale di ciascun blocco all'interno del buffer globale.

Questa scelta permette di gestire al meglio i casi in cui N non è divisibile per Q . I primi $N \% Q$ processi ricevono quindi un elemento extra e *sendcounts/displs* vengono calcolati esplicitamente prima della chiamata per gestire questo caso.

C. Riduzione dei risultati parziali

Dopo che ciascun processo ha calcolato il suo contributo locale:

$$Y^{(p,q)} = A^{(p,q)} X^{(q)}$$

e scaricato il risultato in Y_temp , si esegue una:

```
MPI_Allreduce(Y_temp, Y_local.data, ctx.m * ctx.k,
MPI_FLOAT, MPI_SUM, grid.row_comm);
```

Grazie a questa routine, i contributi parziali di una stessa riga vengono sommati. L'utilizzo di *MPI_Allreduce* combina la riduzione globale di *MPI_Reduce* più il broadcast di una *MPI_Bcast* in una singola operazione collettiva; ogni processo nella stessa riga otterrà il risultato finale, altrimenti con una "classica" *reduce* i processi con indice di colonna diverso da 0 non avrebbero dati di output validi, ma solo parziali.

D. Raccolta del tempo massimo

Viene utilizzata una *MPI_Reduce* per calcolare il wall-clock time della computazione parallela. Poiché i processi possono terminare in momenti leggermente diversi, il tempo complessivo sarà calcolato come il massimo tra tutti i processi, piuttosto che la media, in modo da non sottostimare l'eventuale presenza di ritardi di comunicazione

```
MPI_Reduce(&T_calc, &T_wall, 1, MPI_DOUBLE,
MPI_MAX, 0, MPI_COMM_WORLD);
```

T_wall viene poi utilizzato per calcolare i GFLOPS finali.

E. Gathering di Y

Utilizziamo una *MPI_Gather*:

```
MPI_Gatherv(Y_local->data, Y_local->local_size *
k, MPI_FLOAT, Y_global, recvcunts, displs,
MPI_FLOAT, 0, grid->col_comm);
```

per ricostruire il risultato distribuito. Solo i processi appartenenti alla prima colonna partecipano alla fase di gathering, cioè i “leader di riga”. Analogamente al caso precedente, si usa anche qui la variante “v” per gestire la distribuzione non uniforme delle righe di Y quando M non è divisibile per P.

VI. CUDA RUNTIME E DEVICE MANAGEMENT

6.1 Gestione del Contesto e della Memoria

Ciascun processo MPI inizializza un contesto di esecuzione CUDA indipendente tramite la struttura *CUDAContext*.

La struttura contiene:

- Dimensioni delle matrici locali (m, n, k);
- Device pointers per A, X e Y;
- Un handle cuBLAS.

La device memory allocation viene eseguita durante l’inizializzazione tramite *cudaMalloc*, e tutte le risorse vengono poi deallocate alla fine delle iterazioni tramite *cudaFree* e *cuBLASDestroy*. Questo approccio evita allocazioni di memoria ripetute durante le iterazioni di benchmark e isola la gestione GPU dal layer MPI.

6.2 Trasferimenti Host-To-Device e Device-To-Host

Prima della fase di calcolo, il blocco locale della matrice A e il corrispondente blocco del multi-vettore X vengono trasferiti su device mediante *cudaMemcpy*. I tempi di trasferimento vengono cronometrati separatamente tramite *cudaEventRecord/cudaEventElapsedTime*, permettendo agli overhead di H2D e D2H di essere specificati in maniera indipendente dal tempo di esecuzione del kernel.

- H2D (*cuda_upload*): copia di A_local e X_local da host a device, eseguita una volta sola prima del ciclo;
- Kernel (*cuda_compute*): esecuzione del kernel;
- D2H (*cuda_download*): copia di Y_temp da device a host, ad ogni iterazione.

Come da specifica, il tempo di calcolo T_calc include solo kernel e MPI_Allreduce, escludendo D2H e H2D.

6.3 Selezione Adattiva del Kernel

La funzione *cuda_compute* implementa una politica di dispatch che seleziona il kernel più adatto in base ai valori di k e alla taglia del problema $m * n$. Sono presenti cinque kernel distinti:

- **Register Kernel**, per valori di k minori o uguali a 16: ogni thread calcola una riga intera di Y, accumulando i k risultati parziali nei registri;
- **Warp Kernel**, per valori di k compresi tra 16 (escluso) e 32: un intero warp collabora su una singola riga di Y;
- **Warp Kernel Grid-Stride**, per gli stessi valori di k del caso precedente, ma per carichi di lavoro più grandi (dimensioni locali di A, ovvero $m*n$, maggiori di 32M);
- **Shared memory Kernel**, per valori di k compresi tra 32 (escluso) e 64: approccio tiling con shared memory, blocchi 2D e riduzione degli accessi in memoria globale;
- **Shared memory con 2D Register Tiling e Vectorized Loads**, per valori di k maggiori di 64: approccio basato su register tiling bidimensionale, dove ogni thread calcola un sottoblocco del risultato, favorendo il riuso dei dati nei registri e l’intensità aritmetica, più le vectorized loads, che vanno a ridurre il numero totale di istruzioni e la latenza generale.

6.4 Gestione degli Errori

Tutte le chiamate CUDA vengono wrappate nella macro *gpuErrchk*, che invoca *gpuAssert* controllando il codice di ritorno. In caso di errore, viene stampato un messaggio descrittivo, file e riga sorgente tramite *cudaGetErrorString*.

Senza un wrapper specifico, gli errori “silenziosi” (ad esempio *cudaMalloc* fallita per memoria insufficiente) si propagherebbero fino a produrre risultati errati o crash difficili da diagnosticare.

VII. CUDA KERNELS

In questa sezione esploriamo più in dettaglio i vari kernel implementati.

7.1 Register Kernel

Questo kernel viene usato per valori di k piccoli: in questo regime, l’intero vettore di output associato ad una riga della matrice A può essere mantenuto nei registri, eliminando la necessità di ricorrere alla shared memory e minimizzando il traffico su memoria globale. Ciascun thread è quindi responsabile del calcolo di una completa riga del multi-vettore Y in output, ovvero dei valori $Y[i, 0..k-1]$. Non c’è cooperazione di alcun tipo tra thread, né shared memory, né sincronizzazione: ogni thread lavora in completa autonomia.

In questo kernel, le somme parziali vengono immagazzinate interamente nei registri; il corrispondente array *sums[K]* viene istanziato in tre modi differenti in base al valore di K, parametro noto solo a tempo di compilazione e dipendente dal valore del parametro k del multi-vettore X. K è una costante di compilazione perché in questo modo possiamo sfruttare l’unrolling del loop interno. La direttiva *#pragma unroll* consente, infatti, di espandere il loop interno in K istruzioni separate: così facendo, il compilatore analizza più istruzioni contemporaneamente e questo permette di sovrapporre accessi

alla memoria e operazioni aritmetiche, aumentando quindi l'efficienza complessiva; srotolare i cicli permette anche di avviare più richieste di lettura in parallelo prima che i valori vengano effettivamente consumati dalle ALU.

Per ogni colonna l di A , il thread carica una volta l'elemento $A[i, l]$ e lo riusa per aggiornare tutti e K gli accumulatori. Sia A che X vengono acceduti tramite l'istruzione `__ldg()`, la quale comunica al compilatore di usare la read-only cache della GPU; siccome sia A che X sono read-only durante l'esecuzione del kernel, questo approccio riduce particolarmente la latenza di memoria, alleggerisce la pressione sulle cache L1/L2 e migliora l'utilizzo della banda per i dati che non vengono mai modificati.

In questo kernel, così come in tutti i successivi, viene utilizzata la keyword `__restrict__` per la matrice A e i multi-vettori X e Y . Tale keyword risolve alla radice il problema del “*pointer aliasing*”: è una situazione che si genera nel momento in cui due o più puntatori diversi accedono o fanno riferimento alla stessa area di memoria nello stesso momento, potendo causare comportamenti indefiniti, perché il compilatore assume che ciascun puntatore acceda solo ai propri dati. Il compilatore può quindi assumere, tramite `__restrict__`, che la scrittura attraverso un puntatore non possa modificare dati acceduti da un altro, e l'utilizzo congiunto con `__ldg` massimizza il memory throughput degli operandi read-only.

Al termine del loop, i k risultati vengono scritti in memoria globale. Thread consecutivi scrivono a indirizzi $Y[i*k + j]$ distanziati di k float, quindi la coalescenza ha stride crescente al crescere di k , riducendo progressivamente l'efficienza della coalescenza in scrittura.

7.2 Warp Kernel

Mentre il kernel precedente assegnava un thread per ogni riga, questo assegna un intero warp per ogni riga. Un warp in CUDA è un gruppo di 32 thread, che esegue il codice seguendo il paradigma SIMT (Single Instruction Multiple Threads), secondo cui tutti i thread del warp eseguono lo stesso codice senza essere vincolati dallo stesso path di esecuzione.

Il cambio di strategia rispetto al kernel precedente è motivato dalla crescita di k : per valori superiori a 16, infatti, sarebbero richiesti 17-32 accumulatori per ogni thread; questo aumenterebbe la pressione sui registri stessi e ridurrebbe così facendo l'occupancy. Il warp kernel risolve tale limitazione distribuendo le colonne tra 32 thread.

In particolare, `warp_id` identifica la riga di A (e Y) che il warp deve calcolare, `lane` è la posizione fisica del thread all'interno del warp (0...31) e determina quali colonne di X il thread debba gestire. Infine, l'ultima riga del codice seguente è una “sentinella” volta a proteggere i warp in eccesso nell'ultimo blocco.

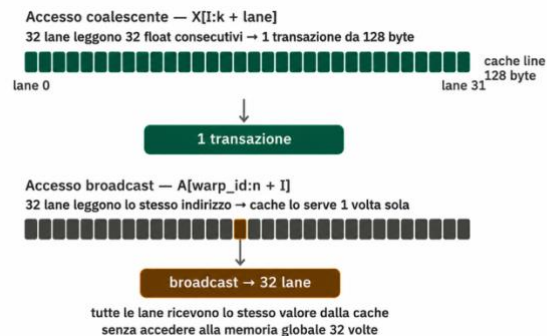
```
int warp_id = (blockIdx.x * blockDim.x + threadIdx.x)
/ WARP_SIZE;
int lane = threadIdx.x % WARP_SIZE;
if (warp_id >= m) return;
```

Ogni warp lane calcola autonomamente e per intero il prodotto scalare tra la riga `warp_id` di A e la colonna `col` di X , accumulando in un singolo registro. Il ciclo for più esterno esegue al più una iterazione seguendo la logica di selezione dinamica dei kernel; questo però rende possibile

l'ipotetico riutilizzo del nucleo di calcolo per valori di k maggiori di 32.

Ogni lane legge indipendentemente $A[\text{warp_id} * n + l]$ ad ogni iterazione l ; la read-only cache permette di effettuare un accesso di tipo broadcast, ovvero l'intero warp leggerà lo stesso indirizzo con una sola transazione verso la memoria globale. Il costo di lettura di A è quindi identico a quello di un singolo thread.

Per il multi-vettore X , ciascuna lane leggerà $X[l*k + col]$; per l fisso, gli indirizzi verranno letti in maniera consecutiva; quindi, il warp genera degli accessi in memoria pienamente coalescenti; una singola transazione da 128 byte soddisfa l'intero warp. Questo perché la GPU non accede alla memoria byte per byte, ma recupera segmenti fissi (cache line) da 128 byte, corrispondente a 32 float consecutivi. Quando un warp esegue un'istruzione di load, l'hardware unifica le 32 richieste in un'unica transazione atomica, massimizzando l'efficienza della banda globale. In questo caso, le 32 lane leggono $X[l*k + lane]$ con $lane = 0, 1, \dots, 31$, cioè esattamente 32 float consecutivi in memoria, che rientrano in una singola cache line.



Per quanto riguarda invece Y , la scrittura finale segue lo stesso pattern contiguo di X : lane consecutive scrivono elementi consecutivi, quindi accesso coalescente.

7.3 Warp-Grid Stride Kernel

Si tratta di una versione alternativa al kernel precedente. Per valori intermedi di k e matrici grandi, la strategia warp-per-row non fornirebbe abbastanza parallelismo per occupare a pieno la GPU; se il numero di righe della matrice A è significativamente maggiore del numero di warp disponibili, molte di queste righe non verrebbero processate finché i precedenti warp non portano a termine il proprio lavoro. La differenza principale non risiede nel calcolo ma nella strategia di lancio sulla GPU. Il kernel precedente infatti lancia esattamente m warps, uno per riga. Per matrici grandi (ad esempio $m*n = 6000 \times 6000$) questo significherebbe migliaia di blocchi; quindi, lo scheduler si ritroverebbe a gestire un numero di blocchi di molto superiore a quello che può eseguire in parallelo, generando overhead di scheduling non trascurabile. Questo approccio risolve il problema lanciando un numero fisso di blocchi, pari a 4 volte il numero di Streaming Multiprocessor del dispositivo, letto a runtime con `cudaDeviceGetAttribute`. Il fattore 4 è un'euristica standard che dà abbastanza warps per SM da nascondere la latenza di memoria, senza creare overhead di scheduling eccessivo.

```
int blocks = 4 * num_sms;
```

Questo kernel implementa il pattern grid-stride: ogni warp, una volta terminato il calcolo della sua riga assegnata, ne preleva una successiva calcolando il nuovo indice tramite il loop esterno, con passo *total_warps*. Si tratta di un tipo di pattern consolidato in ambito CUDA per quei casi in cui l'input è troppo grande rispetto alle risorse fisiche della GPU. Quest'ultima non è quindi più limitata dal numero di blocchi definiti all'avvio, ma continua a processare righe finché non vengono esaurite; il grid diventa quindi una coda di lavoro persistente. In particolare, si ha un bilanciamento del carico totale: se alcuni warp finiscono prima di altri, quelli più veloci si "faranno carico" delle righe rimanenti, assicurando quindi che nessuna risorsa rimanga inattiva fino al completamento dell'ultima riga della matrice. Il corpo interno è lo stesso del kernel precedente, con accesso broadcast per A, accesso coalescente per X e scrittura coalescente su Y.

7.4 Shared Memory Kernel

I kernel precedenti leggono ogni elemento di A e X direttamente dalla memoria globale ad ogni utilizzo. Per valori consistenti di k, tale pattern d'accesso espone il codice a severe inefficienze a causa delle elevate latenze della DRAM globale. Questo kernel risolve il problema introducendo la tecnica del **tiling**: viene suddiviso il dominio in blocchi quadrati di lato *BLOCK_SIZE* e viene caricato ogni blocco una sola volta in shared memory, dove tutti i thread del blocco lo riusano senza dover più accedere alla memoria globale.

```
dim3 block(BLOCK_SIZE, BLOCK_SIZE);
dim3 grid((k + BLOCK_SIZE - 1) / BLOCK_SIZE,
          (m + BLOCK_SIZE - 1) / BLOCK_SIZE);
```

Il singolo thread identificato dalle coordinate locali (*threadIdx.x*, *threadIdx.y*) nel blocco (*blockIdx.x*, *blockIdx.y*) è responsabile del calcolo di un unico elemento di Y.

La griglia copre l'intero multi-vettore Y, con padding di blocchi se le dimensioni non dovessero essere multiple di *BLOCK_SIZE*.

Due buffer di memoria condivisa vengono allocati:

```
__shared__ float A_tile[BLOCK_SIZE][BLOCK_SIZE+1]
__shared__ float X_tile[BLOCK_SIZE][BLOCK_SIZE+1]
```

Andranno ad immagazzinare sottomatrici di A e X. In particolare, il +1 serve a scongiurare il fenomeno dei **bank conflicts**: la shared memory è infatti suddivisa in 32 bank da 4 byte ciascuna; se più thread dello stesso warp tentano di accedere a indirizzi diversi mappati sulla stessa bank simultaneamente, le richieste vengono serializzate. Senza padding, infatti, l'accesso in colonna *A_tile[r][i]* per *r* = 0,...*BLOCK_SIZE*-1 ha indirizzo lineare $r * BS + i$, quindi bank $(r * BS + i) \% 32 = i \% 32$, ovvero tutti i thread leggerebbero la stessa bank i.

Con padding, l'indirizzo diventa $(r * (BS + 1) + i)$, e quindi bank $(r * (BS + 1) + i) \% 32$. Poiché $BS + 1 = 33$ e 33 è coprimo con 32, la sequenza $r * 33 \% 32$ per $r = 0 \dots 31$ è una permutazione degli indici della bank nell'intervallo 0...31, pertanto ogni thread accede a una bank distinta e non si hanno conflitti.

Per ogni iterazione del ciclo di tiling, il blocco di thread carica in shared memory il sotto-blocco di A e il sotto-

blocco di X; nello specifico, ciascun thread carica esattamente un elemento di A e uno di X, distribuendo quindi gli accessi in memoria tra tutti i thread del blocco. Gli accessi sono inoltre coalescenti, in quanto il thread (tx, ty) carica $A[\text{row} * n + a_col]$, dove $a_col = t * BS + tx$; poiché tx varia con continuità all'interno di ogni warp, gli indirizzi di caricamento risultano consecutivi, e lo stesso vale per X.

Dopo aver caricato tutte le tile, i thread vengono sincronizzati per assicurarsi che l'intera tile sia disponibile in memoria condivisa prima dell'inizio del calcolo. Una seconda sincronizzazione garantisce che, prima che i thread sovrascrivano *A_tile* e *X_tile* con la tile successiva, tutti abbiano terminato di usare i valori correnti, scongiurando quindi che i thread più veloci corrompano la tile ancora in uso da thread più lenti.

Il risultato parziale del calcolo viene accumulato nella variabile sum, che funge da accumulatore per il singolo thread; al termine del ciclo, sum conterrà il risultato completo.

Il vantaggio principale di questo approccio risulta essere quindi il riutilizzo dei dati caricati dalla memoria globale: ciascun caricamento da quest'ultima contribuisce a molteplici operazioni floating-point, incrementando l'intensità aritmetica del kernel da 1 dei kernel precedenti a *BLOCK_SIZE*.

7.5 Shared Memory with 2D Register Tiling and Vectorized Loads Kernel

Per valori di k ancora più grandi ($k > 64$), è stata adottata una strategia di ottimizzazione più aggressiva basata su tiling bidimensionale in memoria condivisa, combinato con register blocking e vectorized loads. Questo kernel, quindi, eleva la gerarchia di memoria, consentendo a ciascun thread di calcolare un sottoblocco *TMxTK* (8x4) di elementi di Y, con 8 accumulatori per i risultati parziali *sum[TM][TK]*, che vivono interamente nei registri. Si hanno quindi due livelli di tiling:

- Shared memory: tile *BMxBK* (128x32) del risultato, caricando *BMxBN* (128x32) di A e *BNxBK* (32x32) di X;
- Registri: ogni thread elabora 8x4 elementi della tile tramite un prodotto esterno.

Per saturare la larghezza di banda della memoria globale, il caricamento dei dati da DRAM a shared memory è stato interamente vettorizzato: attraverso il casting dei puntatori a *float4*, ogni thread esegue transazioni di memoria a 128 bit (quindi 4 valori float contigui) in un'unica istruzione hardware, garantendo coalescing della memoria e riducendo notevolmente il numero di richieste hardware totali inviate al memory controller della GPU.

Siccome i trasferimenti dalla memoria che utilizzano *float4* impongono un vincolo stringente di allineamento delle righe a 16 byte, il padding del kernel precedente è stato rimosso, in modo da permettere al compilatore di generare istruzioni di 128 bit allineate.

Uno dei punti critici di questo kernel è l'overhead dovuto al calcolo degli indici di memoria all'interno dei cicli di caricamento della matrice A e multi-vettore X.

A tal scopo, è stato adottato un pre-calcolo esterno per mappare l'identificativo lineare del thread (*linear_tid*) alle

coordinate bidimensionali delle tile condivise (*base_row*, *base_col*), una sola volta fuori dal ciclo principale. Con la configurazione $TM \times TK$ (8×4), ciascun thread legge 8 elementi di A e 4 di X, e grazie ai caricamenti vettorizzati esegue 32 operazioni FMA immediate.

Vengono lanciati 128 thread per ogni blocco; in particolare, per la tile di A in shared memory, si hanno $128 \times 32 = 4096$ float, pertanto 1024 float4. Lo scheduling interno prevede la partizione del lavoro in un numero fisso di turni per consentire ai thread di coprire l'intera area; tale parametro è definito dalla costante: $STRIDES_A = 1024 / 128 = 8$. Il fatto che quest'ultima sia una costante a tempo di compilazione permette al compilatore di effettuare l'unrolling statico del loop di caricamento, trasformando le 8 iterazioni in 8 istruzioni sequenziali. Per X avviene lo stesso, ma $STRIDES_X$ sarà pari a 2, in quanto si hanno solo 256 vettori float4.

All'interno del loop di caricamento, quindi, il calcolo della riga si riduce ad una semplice operazione di accumulo lineare.

Una volta caricati i dati in shared memory e sincronizzati i thread, il kernel avvia la fase di calcolo puro. Prima di eseguire i prodotti, i frammenti necessari di A e X vengono copiati nei registri locali *reg_A[TM]* e *reg_X[TK]*; il ciclo interno esegue un prodotto tra i registri caricati, accumulando i risultati in *sum*, e tutti i cicli sono preceduti dalla direttiva *#pragma unroll*, utilizzata per forzare il compilatore ad eliminare il controllo dei loop, espandendo le operazioni in una sequenza lineare di istruzioni.

Alla fine del loop di calcolo, ciascun thread scrive i propri risultati in Y, applicando i boundary check opportuni per gestire i blocchi sul bordo della matrice quando le dimensioni m e n non sono multipli esatti di BM o BK. Questo kernel combina quindi diverse tecniche di ottimizzazione, riducendo di molto il traffico verso la memoria globale e facendo sì che la GPU spenda una frazione di tempo più ampia nel calcolo piuttosto che nell'attesa dei dati, rendendo quindi il kernel prettamente compute-bound anziché memory-bound.

VIII. CUBLAS BASELINE

È stata implementata una versione di riferimento ideale usando la libreria cuBLAS di NVIDIA. Visto che cuBLAS fornisce delle routine altamente ottimizzate per nuclei di calcolo che includono matrici dense/sparse, l'implementazione considerata funge da upper bound teorico e pratico delle prestazioni del prodotto tra matrice e multi-vettore.

In particolare, il prodotto viene eseguito mediante la routine GEMM *cublasSgemv()*. cuBLAS richiede che le matrici siano rappresentate nel formato column-major, ovvero gli elementi della matrice vengono salvati colonna per colonna in maniera sequenziale; pertanto, è stata sfruttata l'identità algebrica della trasposizione del prodotto matriciale, al fine di evitare l'overhead computazionale oneroso di una trasposizione fisica dei dati in memoria globale:

$$Y^T = (A * X)^T = X^T * A^T$$

Nel codice, questo comportamento si riflette nell'inversione dell'ordine dei fattori passati alla funzione: il ruolo di prima

matrice operanda è assegnato a d_X (con dimensione interna k); quello di seconda matrice operanda è assegnato a d_A (con dimensione interna n) ed entrambe le matrici vengono passate con il flag *CUBLAS_OP_N* (non trasposte), lasciando che sia la lettura logica intrinseca del layout column-major a operare la trasposizione geometrica.

Il calcolo del tempo di esecuzione viene isolato attraverso l'uso di CUDA Events; siccome la matrice è distribuita tra vari processi MPI, ciascuno di essi calcola solo il suo contributo locale; quindi, il tempo totale di esecuzione cuBLAS considera anche l'accumulo dei risultati parziali tramite la primitiva *MPI_Allreduce*, garantendo così un metro di paragone equo con l'implementazione CUDA.

IX. CONFIGURAZIONI TESTATE

Durante la fase di benchmark sono state valutate diverse configurazioni di M, N e k. In particolare, i valori di k testati sono esattamente dieci:

$$k \in \{3, 6, 8, 16, 20, 32, 64, 96, 100, 128\}$$

Questi valori sono stati selezionati al fine di coprire tutte le regioni di dispatch per i vari kernel implementati. Sette configurazioni di matrice A sono state prese in considerazione:

Configurazione	Dimensioni	Scopo
Small square	1024x1024	Baseline
Square	8000x8000	M = N
Large square	12000x12000	M = N (large)
Power-of-two square	8192x8192	Dimensioni GPU-friendly
Tall	12000x4000	M = 3N
Very Tall	24576x8192	M = 3N (large)
Wide	4000x8000	N = 2M

Queste configurazioni sono state scelte per verificare come la performance dipenda dall'aspect ratio della matrice e dalla taglia totale del problema; le matrici con dimensioni basate su potenze di due tipicamente garantiscono un maggiore allineamento in memoria, miglior coalescing e meno boundary checks.

In totale, le matrici selezionate coprono 3 regimi distinti, quali M = N, M = 3N e N = 2M, sottolineando differenti workload computazionali e pattern di accessi in memoria. Ciascuna configurazione è stata ripetuta per un numero totale di iterazioni pari a dieci e, di conseguenza, le misurazioni temporali corrispondono al tempo medio di esecuzione su tutte le iterazioni.

X. RACCOLTA E ANALISI DEI RISULTATI

I benchmark sono stati eseguiti sul server di dipartimento equipaggiato con GPU NVIDIA Quadro RTX 5000, e i risultati raccolti su un file CSV; sono stati successivamente generati dei grafici esplicativi a partire dal dataset filtrato tramite le librerie Python di visualizzazione scientifica, quali Matplotlib e Seaborn.

10.1 GFLOPS vs K

Per ogni valore di M ed N fissi, è stato generato un grafico che mette in luce, al variare del parametro k, i GFLOPS ottenuti dai vari kernel implementati, con un confronto immediato con la baseline cuBLAS. Lo sfondo è suddiviso in regioni ombreggiate corrispondenti ai range di validità o efficienza dei diversi stadi architetturali.

Il grafico è in realtà diviso in due sezioni; oltre alla prima già menzionata, la seconda sezione mostra l'efficienza percentuale normalizzata rispetto all'upper bound di cuBLAS.

In particolare, per matrici di bassa dimensionalità (1024x1024) [Figura 1], si può notare come i kernel custom superino costantemente la baseline in ogni regime testato, registrando il picco assoluto di efficienza che supera il 340%; questo dimostra che la launch latency di cuBLAS, su matrici piccole, domina il tempo di calcolo puro, rendendo i kernel custom più "adatti" a massimizzare il throughput in quanto privi di overhead di astrazione, logiche di partizionamento o euristiche di runtime che caratterizzano la libreria NVIDIA.

Per matrici di medie dimensioni come la 4000x8000 [Figura 2], notiamo invece come cuBLAS cominci ad ammortizzare i propri overhead, riuscendo ad ottenere un picco di GFLOPS di più di 4000/s per k = 128. Il kernel basato su registri soffre l'aumento delle dimensioni, degradando dopo k = 8, e mostrando quindi una pressione sui registri considerevole; I registri all'interno di uno Streaming Multiprocessor sono una risorsa fissa e limitata. Se un singolo thread richiede più registri, l'SM è costretto a ospitare meno warp contemporaneamente (riduzione dell'occupancy). Il kernel basato su warp si dimostra invece particolarmente robusto, riuscendo a toccare il 96% di efficienza relativa per k = 32, e il kernel shared memory/register tiling mostra un throughput assoluto di oltre 3097 GFLOPS/s e un'efficienza relativa del 99% per k = 96. Si può notare anche una leggera fluttuazione dei GFLOPS passando da k = 96 a k = 100 e successivamente a k = 128; essendo le dimensioni delle tile fisse, alcuni valori di k si adattano meglio rispetto ad altri. 96 e 128 sono infatti multipli perfetti di BK (= 32), mentre k = 100 non lo è, pertanto, la non-divisibilità per le dimensioni del blocco costringe il compilatore ad attivare rami condizionali per evitare accessi fuori memoria, causando warp divergence cronica e lasciando molti thread inattivi; viene richiesta, infatti, un'ulteriore tile finale parzialmente occupata; ciò porta a thread inattivi, boundary checks aggiuntivi e generale calo di performance.

Per matrici quadrate di grandi dimensioni (come 8000x8000, 8192x8192) [Figura 3,4] i grafici presentano profili prestazionali e tendenze quasi identici. cuBLAS si stabilisce intorno ai 5600-5900 GFLOPS/s.

Otteniamo una performance leggermente migliore per il register kernel, sebbene l'efficienza risulti leggermente peggiore in quanto la semplice strategia di un thread per riga non sfrutta il riutilizzo dei dati tra thread e scala meno efficacemente rispetto a cuBLAS. L'ultimo custom kernel, per k = 96, mostra un picco di quasi 4000 GFLOPS/s per un'efficienza del 93%, da attribuire sicuramente all'allineamento perfetto tra le dimensioni del problema e le

taglie delle tile, che portano ad una massima utilizzazione delle risorse computazionali.

Per k = 128 notiamo invece un lieve calo della performance, da attribuire a una maggiore pressione sui registri e ad un peggior riutilizzo della cache L2; infatti, per k = 96, una buona porzione di dati rimane residente nella cache stessa, ma per k = 128 il volume totale ne supera la capacità critica.

Per matrici grandi e rettangolari, come la configurazione 12000x4000 [Figura 5], notiamo un comportamento pressoché simile ai casi precedenti, se non un quasi 100% di efficienza rispetto a cuBLAS per valori piccoli di k.

Per la controparte quadrata (12000x12000) [Figura 6], invece, viene selezionato il kernel Warp GS, che tuttavia presenta una modesta performance, a segnalare il fatto che il problema è ora diventato memory-bound: la matrice infatti è molto grande e ciò genera continue cache miss nelle cache L1/L2; lo "sblocco" prestazionale avviene solamente con l'ultimo kernel il quale, grazie a memoria condivisa e tiling nei registri, riduce drasticamente l'indice di comunicazione con la memoria globale e permette alle prestazioni di aumentare fino a più del 86% di efficienza, e un tetto massimo di 4718 GFLOPS/s per k = 128.

L'ultimo caso analizzato è un test di stress computazionale realizzato tramite la configurazione di una matrice 24576x8192 [Figura 7]. cuBLAS riesce a raggiungere un picco di oltre 9000 GFLOPS/s; l'ultimo custom kernel risulta comunque essere il migliore, scalando fino a 4074 GFLOPS/s. Questo esempio mostra che, all'aumentare della taglia del problema, l'assenza di un modello ottimizzato basato su registri e shared memory rende il codice insostenibile. Il kernel basato su memoria condivisa e register tiling bidimensionale riesce infatti a convertire un problema strutturalmente memory-bound in un regime compute-bound.

10.2 Speedup vs K

In tutti i grafici generati [Figure 10/16] si può notare un ovvio speedup crescente a partire da bassi valori di k fino ad arrivare al massimo di 128. Si ha tuttavia un calo esattamente in corrispondenza di k = 100, e questo è dovuto sempre al fenomeno di warp divergence.

Nell'ultimo custom kernel, la grid dimension sull'asse x è determinata da

$$\left\lceil \frac{k}{BK} \right\rceil$$

Per k = 96 abbiamo 3 blocchi esatti e per k = 128 4 blocchi esatti; quindi, la GPU lavora al 100% delle sue capacità.

Per k = 100, invece, per colpa di 4 elementi extra, dobbiamo lanciare una quarta colonna intera di thread block, che andrà a gestire le colonne da 96 a 127, ma dato che k è 100, solo 4 colonne su 32 sono valide; quindi, il restante 87% del blocco rimane inutilizzato. Questo bottleneck di asimmetria penalizza il tempo di esecuzione totale rispetto ai casi allineati.

10.3 Time Breakdown Analysis

Il grafico Time Breakdown [Figura 17] decompone il tempo di esecuzione in quattro componenti:

- Tempo di calcolo sulla GPU
- MPI AllReduce

- Trasferimento Host-to-Device
- Trasferimento Device-to-Host

Si può notare chiaramente che il trasferimento H2D domina, rappresentando in alcuni casi più del 70-80% del tempo di esecuzione totale, indicando quindi come principale bottleneck i trasferimenti PCIe, mentre i custom kernel hanno un tempo di esecuzione relativamente rapido una volta che i dati si trovano sulla GPU.

Si nota anche un generale aumento del tempo di calcolo sulla GPU all'aumentare di k , in accordo con il costo computazionale di $O(MNk)$. L'overhead dovuto a MPI è invece minimo, quindi anche per matrici più grandi, l'approccio di decomposizione distribuita e row-wise risulta efficace.

I trasferimenti D2H sono, anch'essi, trascurabili, in quanto il multi-vettore $Y = M \times k$, e k è decisamente più piccolo di N ; nei trasferimenti H2D abbiamo una matrice decisamente più grande, ma anche un ulteriore multi-vettore da trasferire su GPU.

Il grafico mostra, inoltre, solo casi di matrici quadrate per semplificare la visualizzazione complessiva.

10.4 Roofline-style Analysis

Questo grafico [Figura 8] mette in relazione l'intensità aritmetica (FLOPs/byte) con la performance (GLOPS/s), utilizzando scale logaritmiche in modo da comprimere i dati in intervalli più gestibili numericamente.

L'intensità aritmetica misura il rapporto tra le operazioni in virgola mobile e la quantità di dati letti o scritti in memoria. Un'alta intensità aritmetica corrisponde generalmente a un miglior utilizzo delle risorse della GPU.

L'osservazione chiave che si può trarre dal plot è che un aumento della performance corrisponde ad un aumento dell'intensità aritmetica. Spostandosi da sinistra verso destra nel grafico, infatti, si nota come i carichi di lavoro a bassa intensità raggiungano "solo" poche centinaia di GFLOPS/s, mentre i carichi di lavoro ad alta intensità arrivano a diversi TFLOPS/s.

I kernel che si appoggiano principalmente su accessi in memoria globale, come il register o il warp/warp GS kernel, restano nelle regioni più basse della performance, mentre i kernel che si appoggiano su shared memory e tiling hanno una più alta intensità aritmetica e un throughput sostanzialmente maggiore, dimostrando ancora una volta l'importanza del riutilizzo dei dati e di ottimizzazioni relative alla gerarchia di memoria.

Nel grafico viene mostrato anche il risultato ottenuto da cuBLAS; si può nuovamente notare come il kernel 2D reg-tiling con shared memory riesca a raggiungere una sostanziale frazione della performance della libreria NVIDIA.

10.5 Scaling curves

Il grafico rappresenta i GFLOPS/s in funzione della dimensione totale del problema ($M \times N \times k$) [Figura 9]. Si può notare come la performance migliori all'aumentare progressivo delle dimensioni del problema; si ha infatti un'occupancy maggiore, un ammortizzamento degli accessi in memoria e in generale un utilizzo complessivo migliore della GPU.

Il grafico mostra quanto si è già detto in relazione ai singoli kernel, ovvero buona performance per piccoli valori di k ma scalabilità limitata per il register kernel; maggiore scalabilità

e performance nei kernel warp/warp GS, e i kernel basati su shared memory si confermano i migliori a livello di performance, sempre per motivi di riutilizzo dei dati in memoria condivisa e/o registri, e minor pressione sulla banda di memoria.

In generale, le dimensioni del problema non sono un indicatore univoco della performance, ma anche la geometria delle matrici (l'aspect ratio) e l'allineamento ai multipli del warp giocano un ruolo fondamentale. I punti che nel grafico "crollano" verso il basso corrispondono, infatti, alla configurazione caratterizzata dal problema del warp divergence/grid tail effect. Persino cuBLAS esibisce tali oscillazioni.

XI. CONCLUSIONI

I risultati ottenuti mostrano, complessivamente, come lo sfruttamento progressivo della gerarchia di memoria della GPU porti a miglioramenti sempre più sostanziali della performance. Si potrebbe pensare di ottenere dei risultati ancora migliori, per esempio:

- sfruttando la tecnica di overlapping tra comunicazione e calcolo, tramite i CUDA streams;
- usando la memoria pinned per trasferimenti di dati più veloci;
- ricorrendo a tecniche di double buffering da aggiungere all'ultimo kernel, per rendere il caricamento delle tile e il prodotto esterno "concorrenti".

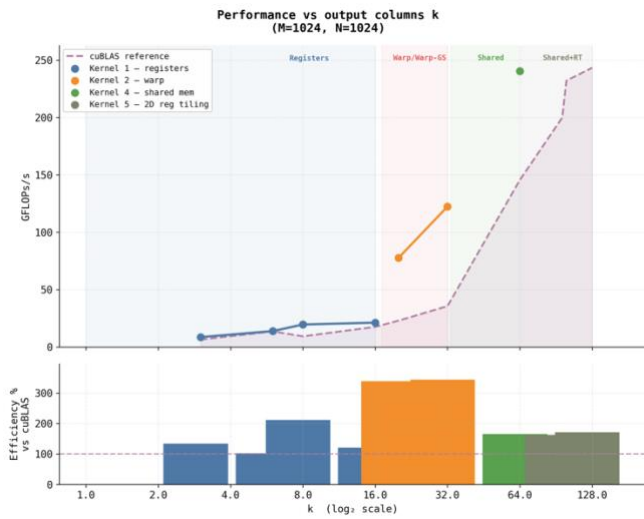


Figura 1: GFLOPS vs k ed Efficiency (1024x1024)

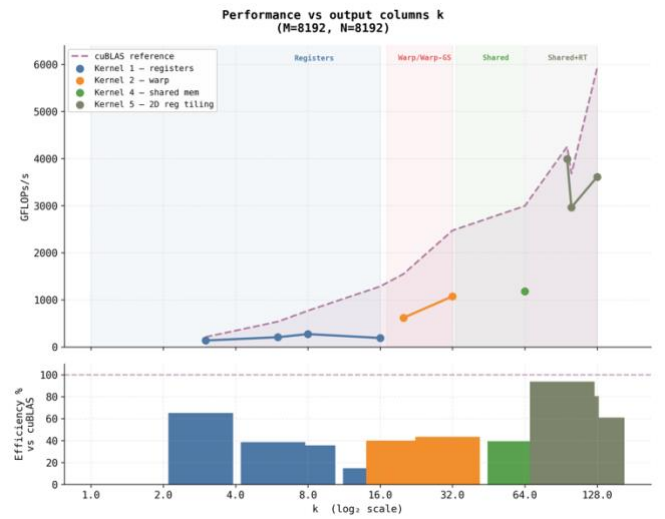


Figura 4: GFLOPS vs k ed Efficiency (8192x8192)

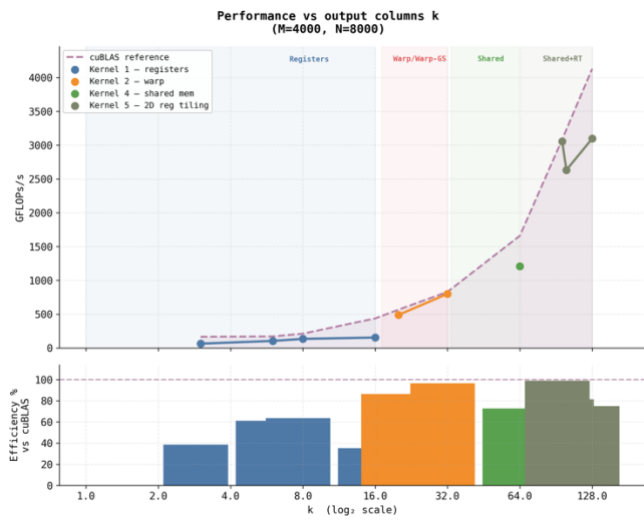


Figura 2: GLOPS vs k ed Efficiency (4000x8000)

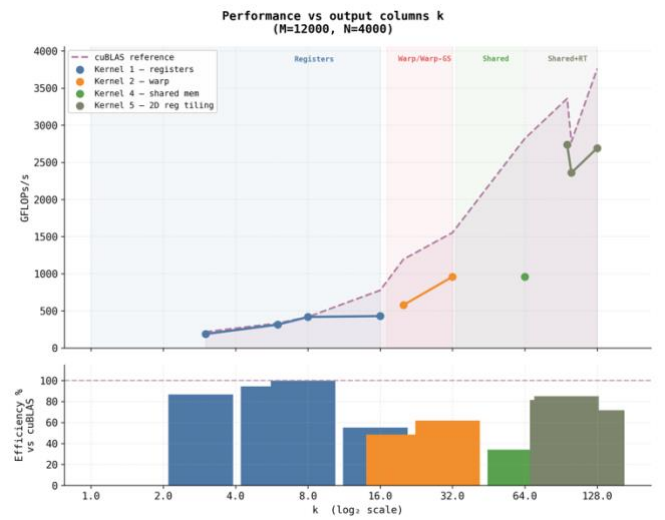


Figura 5: GFLOPS vs k ed Efficiency (12000x4000)

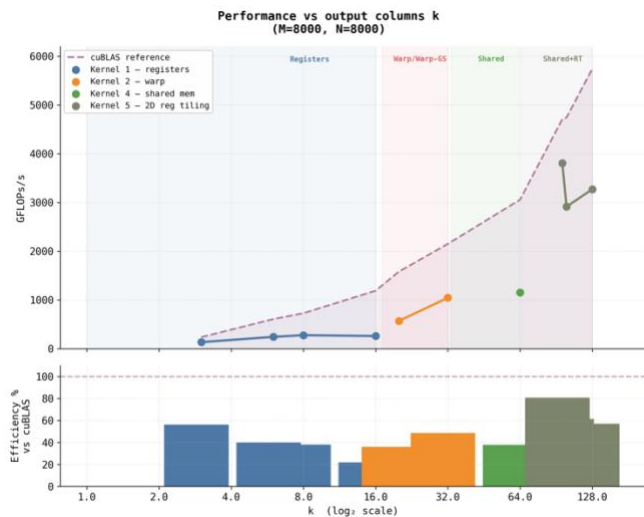


Figura 3: GFLOPS vs k ed Efficiency (8000x8000)

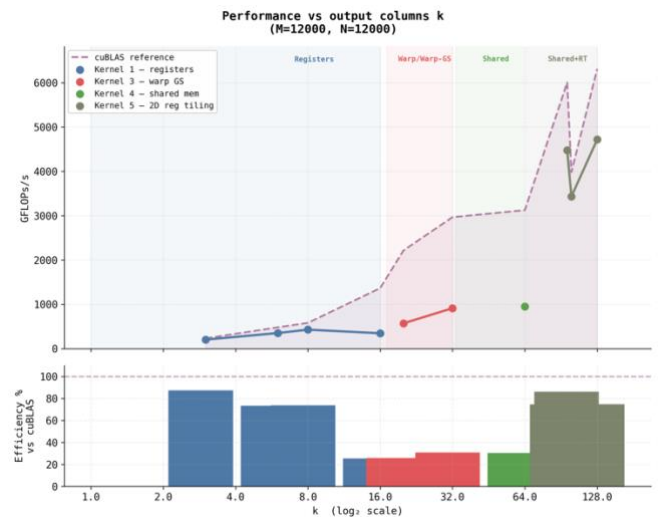


Figura 6: GFLOPS vs k ed Efficiency (12000x12000)

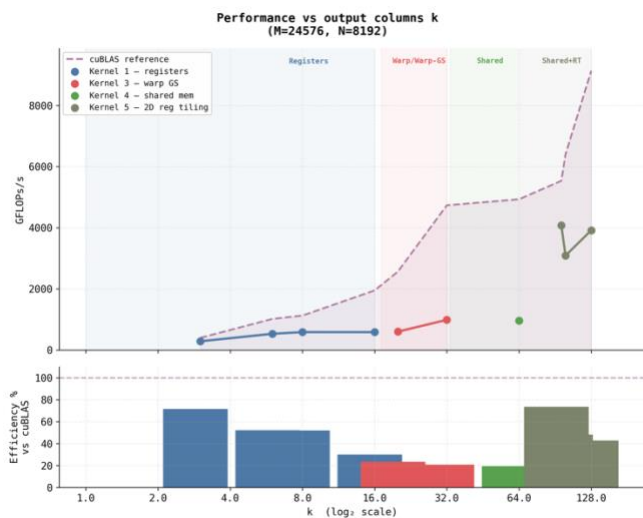


Figura 7: GFLOPs vs k ed Efficiency (24576x8192)

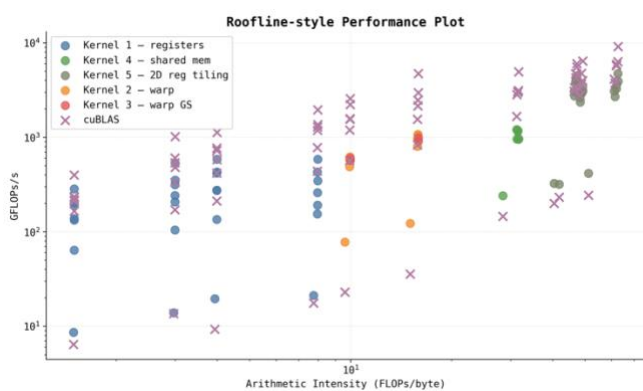


Figura 8: Roofline-style Analysis

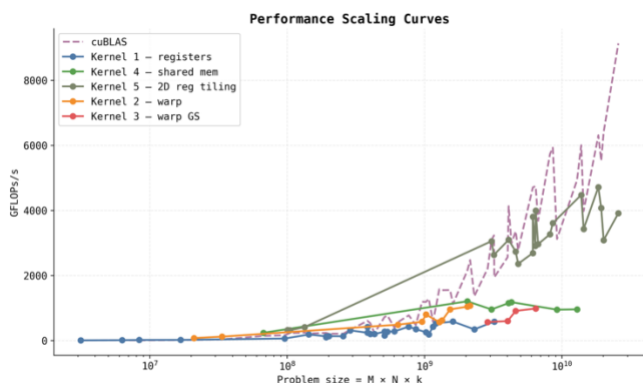


Figura 9: Scaling curves

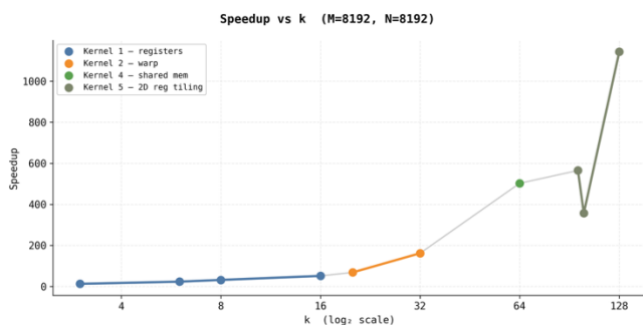


Figura 10: Speedup vs k (8192x8192)

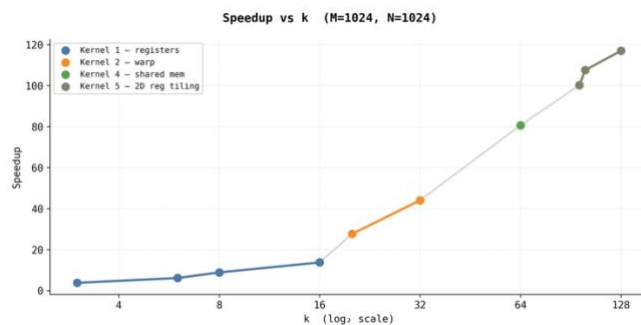


Figura 11: Speedup vs k (1024x1024)

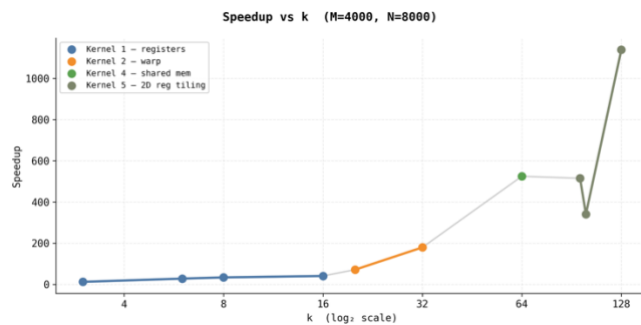


Figura 12: Speedup vs k (4000x8000)

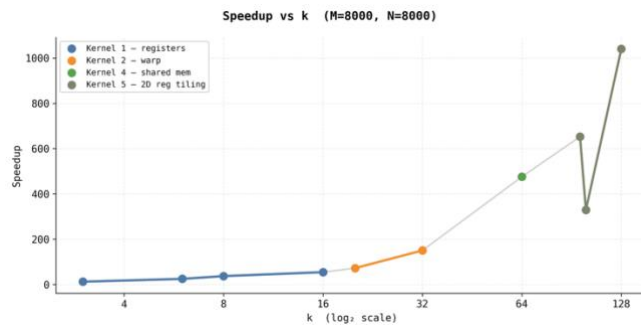


Figura 13: Speedup vs k (8000x8000)

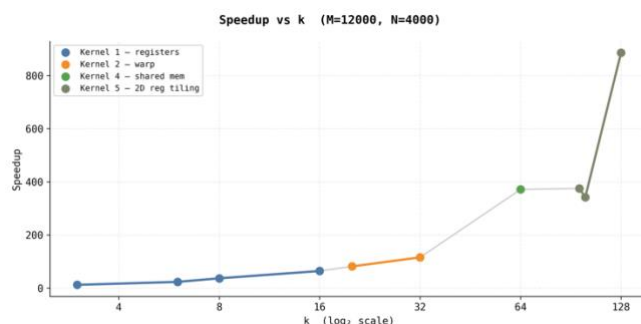


Figura 14: Speedup vs k (12000x4000)

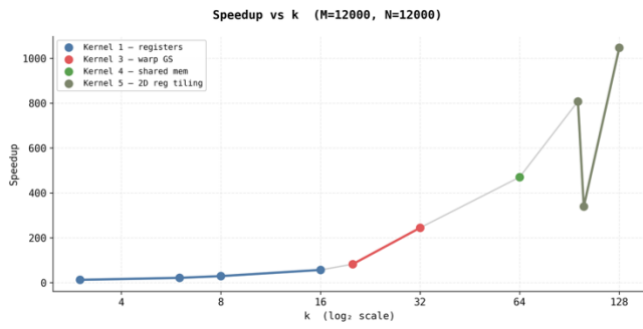


Figure 15: Speedup vs k (12000x12000)

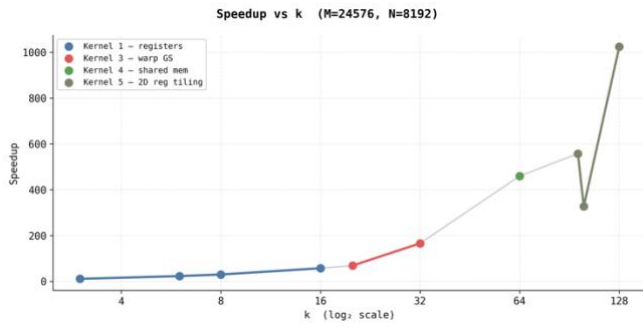


Figure 16: Speedup vs k (24576x8192)

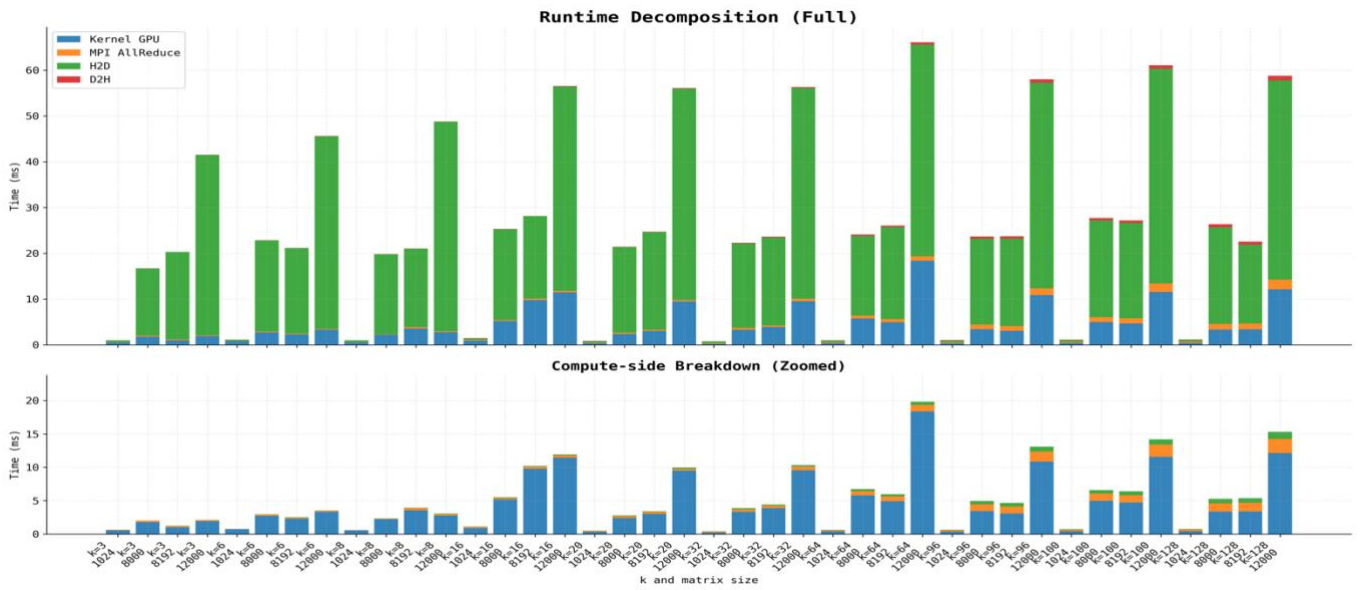


Figure 17: Time Breakdown Analysis